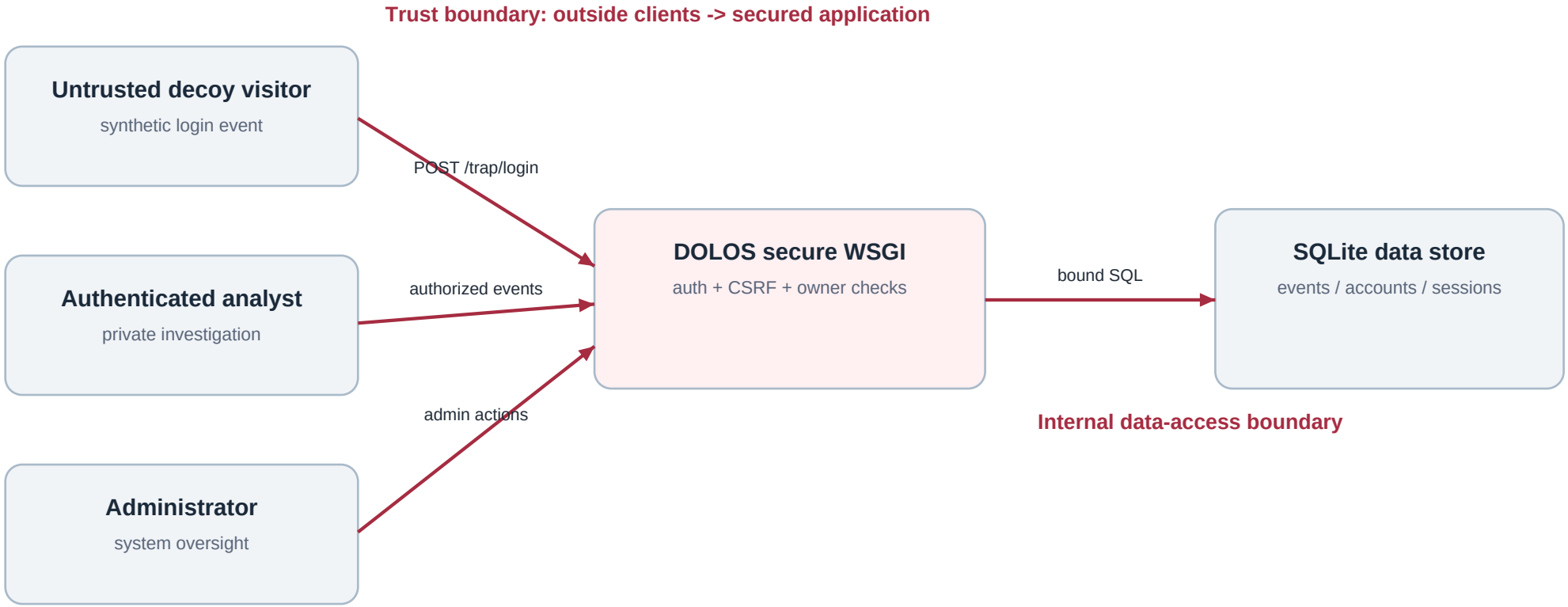
CONTEXT / SYSTEM SCOPE

# Security Threat Model

DOLOS is a local-first deception monitoring prototype. The secured application stores synthetic decoy events and supports authenticated, owner-scoped investigations. An intentionally unsafe localhost-only demonstration is excluded from deployment.

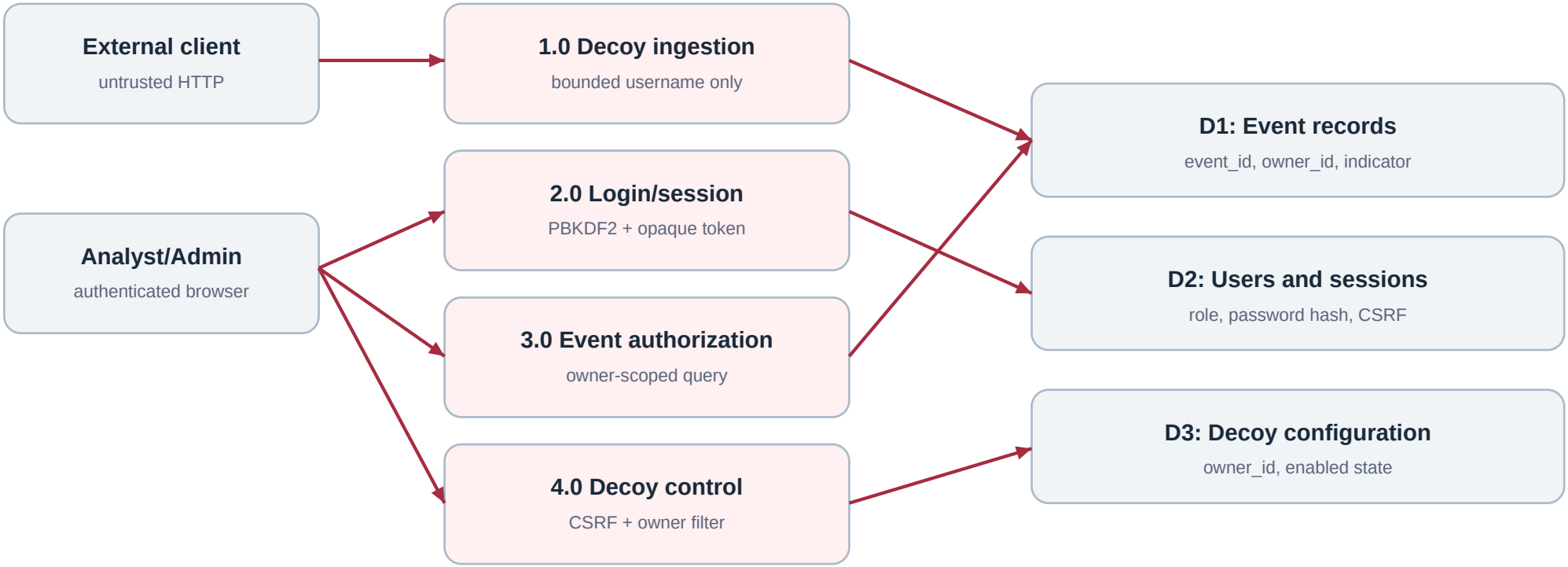



**Trust boundaries:** The external browser/decoy interface is untrusted. The application authenticates analyst requests before SQL access. Native memory-unsafe behavior is only present inside the separate teaching lab.

FLOW DECOMPOSITION

# Level-1 Data-Flow Diagram

Five logical operations: decoy ingestion, authentication, incident authorization, decoy control and investigation. D1 stores events, D2 stores users/sessions, D3 stores decoy states. Untrusted fields are bounded, escaped and passed to parameterized SQL.




**5.0 Investigation: parameterized event search within D1 + D2**

**CIA link:** private ownership checks protect Confidentiality; CSRF and access-control checks protect Integrity; bounded inputs, health checks and container limits support Availability.

THREATS BY DFD ELEMENT

# STRIDE Analysis

| Element | Spoofing | Tampering | Repudiation | Disclosure | DoS | Elevation / control |
|---|---|---|---|---|---|---|
| Decoy ingress | Fake identities | Injected indicators | Forged claims | Echoed data | Flooding | Unauthorized decoy action |
| Authentication | Stolen session | Cookie change | Missing login audit | Hash/token leakage | Login floods | Session hijack |
| Incident retrieval | Account spoof | ID tampering | No read audit | T-IDOR | Query overload | Admin bypass |
| Decoy controls | Forged analyst | T-CSRF | Unlogged toggles | State leak | Disable decoy | Owner bypass |
| Investigation | Fake identity | T-CMD | Missing audit | SQL leakage | Expensive search | Shell RCE |
| Native parser | Malicious tag | T-BOF | Lost test trace | Memory disclosure | Crash | Control flow overwrite |
| Browser UI | UI impersonation | Clickjacking | Lack of user proof | T-FRAME / XSS | Client floods | Coerced actions |
| GitHub CI | Forged committer | Malicious build | Missing review | Secret leakage | Blocked build | Action misuse |

**Applied controls:** salted password hashes; HttpOnly/SameSite session cookies; per-session CSRF; owner-scoped queries; no shell evaluation; validated C buffers; CSP frame-ancestors none; X-Frame-Options DENY; review-gated CI. Remaining controls include rate limits, immutable audit logs, TLS and production deployment hardening.

ATTACK GOALS AND AND/OR PATHS

# Two Attack Trees

## Attack tree A - Goal (OR): breach data ownership or modify decoy settings

| Path | AND prerequisites | Mitigation |
|---|---|---|
| A1 / IDOR | Acquire analyst session -> guess another event ID -> no owner check -> foreign event read | WHERE id=? AND owner_id=?; 404 for unauthorized |
| A2 / CSRF | Victim authenticated -> visits malicious page -> forged POST -> missing CSRF token | Unpredictable synchronizer token + SameSite |
| A3 / Frame | Attacker-controlled page -> embed sensitive admin frame -> induce click | CSP frame-ancestors none + X-Frame-Options DENY |

## Attack tree B - Goal (OR): compromise event processing

| Path | AND prerequisites | Mitigation |
|---|---|---|
| B1 / Command | Reach investigation endpoint -> shell interpolation of input -> injected separator executes | Bound SQL; no shell invocation |
| B2 / Overflow | Reach native metadata parser -> send >=16-byte tag -> unbounded strcpy overflows stack | Check length, snprintf, ASan, canary |
| B3 / XSS | Submit hostile tag -> store event -> template inserts unescaped HTML -> script runs | HTML escaping and restrictive CSP |

These are structured hypothetical attack paths using synthetic data. The isolated demonstrations prove specific paths/conditions; they do not prove arbitrary code execution on production systems.

QUANTITATIVE RISK REGISTER

# CVSS v3.1 & Residual Risks

| ID | Risk | CVSS v3.1 Base vector | Score | Response |
|---|---|---|---|---|
| T-BOF | Native overflow | AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H | 7.8 | Bound C input + ASan |
| T-CMD | Shell injection | AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H | 8.8 | Replace shell with SQL |
| T-IDOR | Unauthorized ID | AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N | 8.1 | Owner-scoped SQL |
| T-CSRF | Forged toggle | AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:L | 7.1 | Session CSRF token |
| T-FRAME | Coerced UI click | AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N | 6.5 | CSP + XFO |

**Scoring assumptions:** Base score considers a hypothetical unsafe implementation with network-accessible analyst interface. Read-only IDOR proof is narrower than the modelled C+I impact. Frame embedding alone is narrower than a successful coerced UI action. Scoring follows FIRST CVSS v3.1. Generate the complete vectors and values with scripts/cvss31.py.

## Residual risks after fixes

- Reference WSGI is not internet-grade; TLS/reverse proxy and public deployment review required.
- No rate limiting, SIEM integration or immutable audit; repeated decoy requests may exhaust resources.
- SQLite database is not encrypted or replicated; file theft, disk failure and tampering remain possible.
- Container non-root and scanner gates lower risk but cannot guarantee absence of vulnerabilities.

**Traceability:** PNE_Report.md -> T-* risks -> docs/JIRA_BACKLOG.md -> SECURITY_IMPLEMENTATION.md -> unit tests -> GitHub PR + evidence screenshots. The final approval, scans, human PR reviews and 18-30 page report are user-run deliverables.